DATA ANALYSIS INTERNSHIP

DAY 13 REPORT – Matplotlib Charts and Visualization

Title

Visualization of Different Charts Using Python Matplotlib Library

---

Introduction

In today's session, different types of charts and plots were created using the Python Matplotlib library. The objective of this task was to understand how various visualization techniques help in representing numerical and mathematical data effectively. Multiple graphs such as line plots, bar charts, scatter plots, fill-between plots, stem plots, and combined charts were generated and analyzed.

Program

```
import numpy as np

import matplotlib.pyplot as plt


# Data

x = np.linspace(0, 10, 50)

y = np.sin(x)

z = np.cos(x)


# Sin Line Plot

plt.subplot(2,2,1)

plt.plot(x, y)
```

```
plt.title("Sin Line")

# Cos Step Plot
plt.subplot(2,2,2)
plt.step(x, z)
plt.title("Cos Step")

# Fill Between
plt.subplot(2,2,3)
plt.fill_between(x, y, alpha=0.7)
plt.title("Fill Between")

# Stem Plot
plt.subplot(2,2,4)
plt.stem(np.arange(50), np.random.rand(50))
plt.title("Stem Plot")

plt.show()
```

Functions Used

Function	Purpose
`np.linspace()`	Generates evenly spaced values
`np.sin()`	Computes sine values
`np.cos()`	Computes cosine values

`plt.plot()`	Creates line chart
`plt.step()`	Creates step graph
`plt.fill_between()`	Fills area between graph and axis
`plt.stem()`	Creates stem plot
`plt.bar()`	Creates bar chart
`plt.scatter()`	Creates scatter plot
`plt.subplot()`	Displays multiple plots in same figure
`plt.title()`	Adds title to graph
`plt.show()`	Displays the output

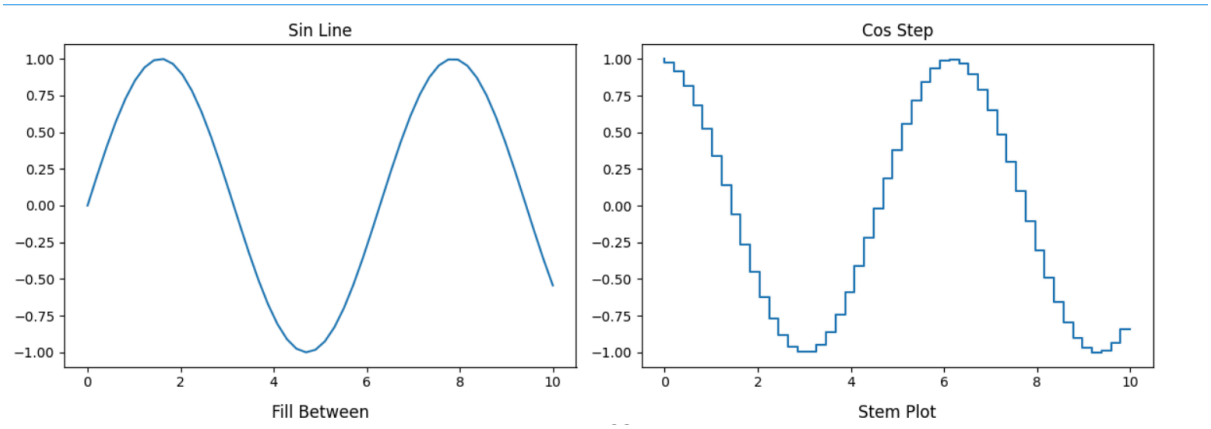
Output Description

1. Sin Line Plot

The sine wave was displayed using a line graph. It shows smooth periodic oscillation between -1 and 1.

2. Cos Step Plot

The cosine values were represented using a step graph where data changes appear in staircase form.

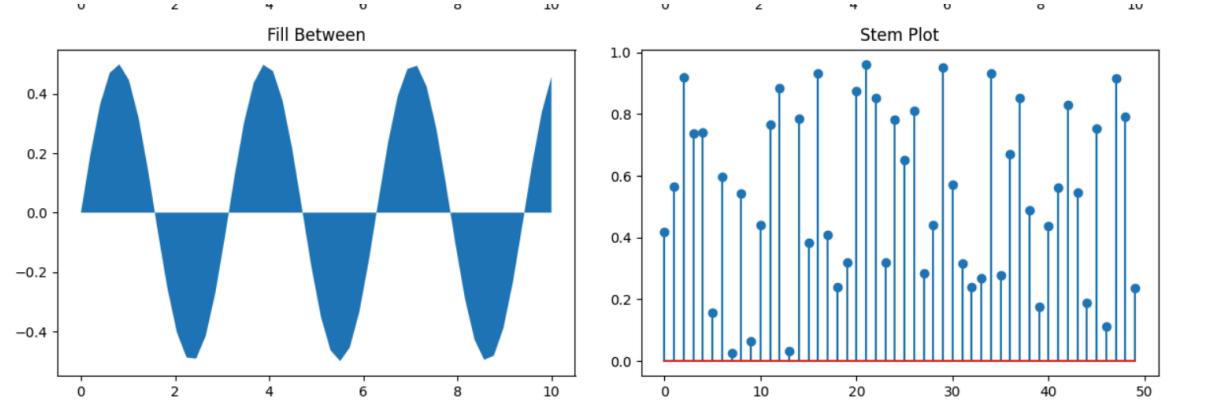


3. Fill Between Plot

The area between the sine curve and x-axis was filled with color to improve visual representation.

4. Stem Plot

The stem plot displayed discrete random values using vertical lines and markers.



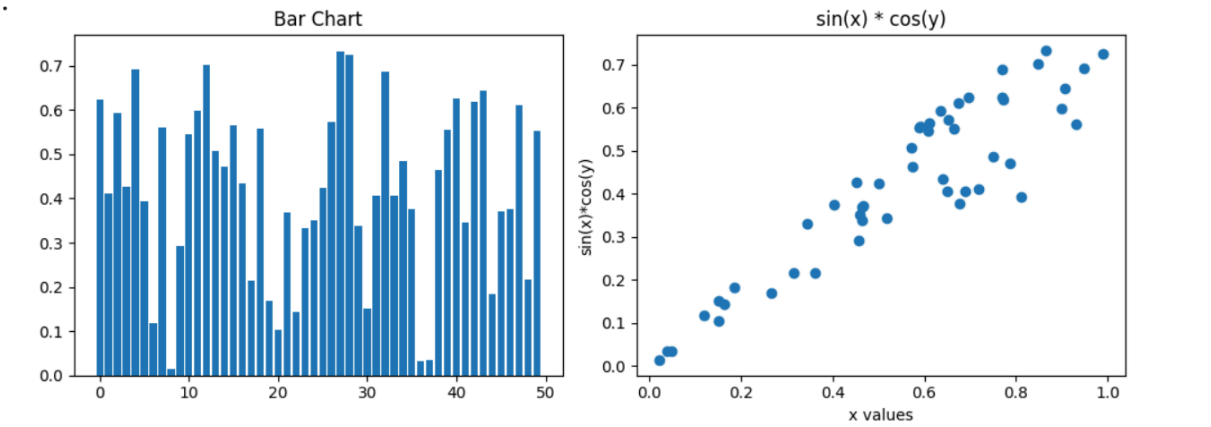
5. Bar Chart

The bar chart represented random numerical values using rectangular bars for comparison.

6. Scatter Plot

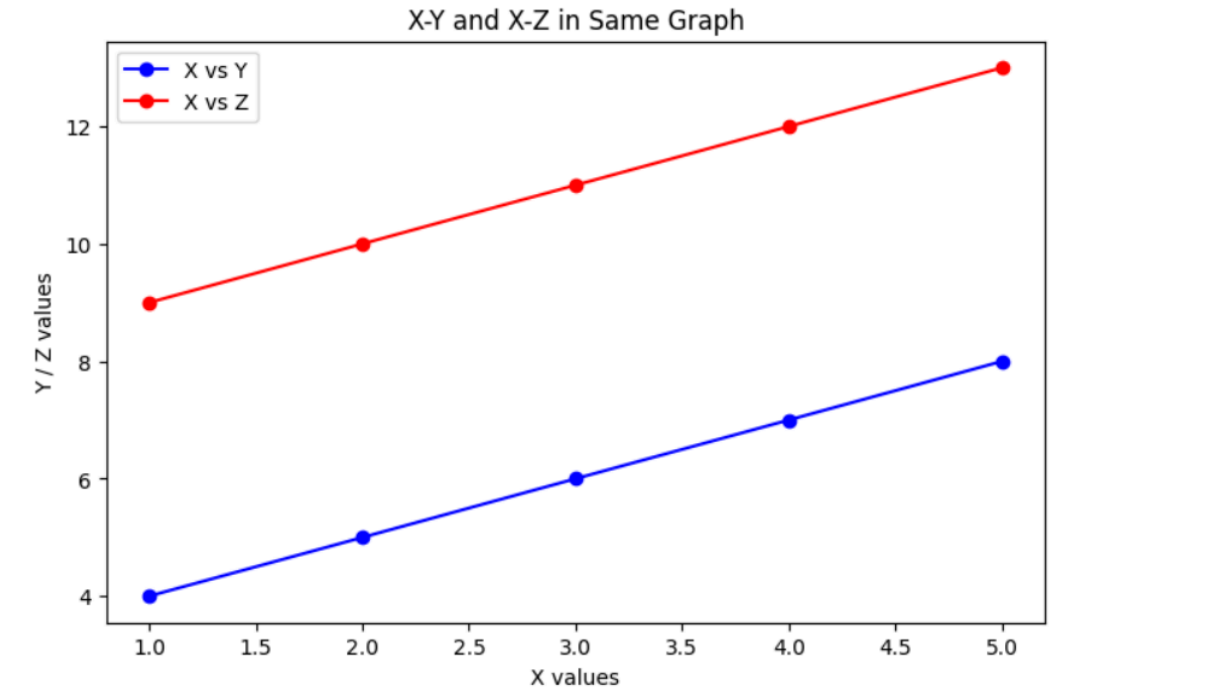
The scatter plot displayed relationships between x values and `sin(x) * cos(y)` values using points.

```
plt.show()
```



7. Combined Line Graph

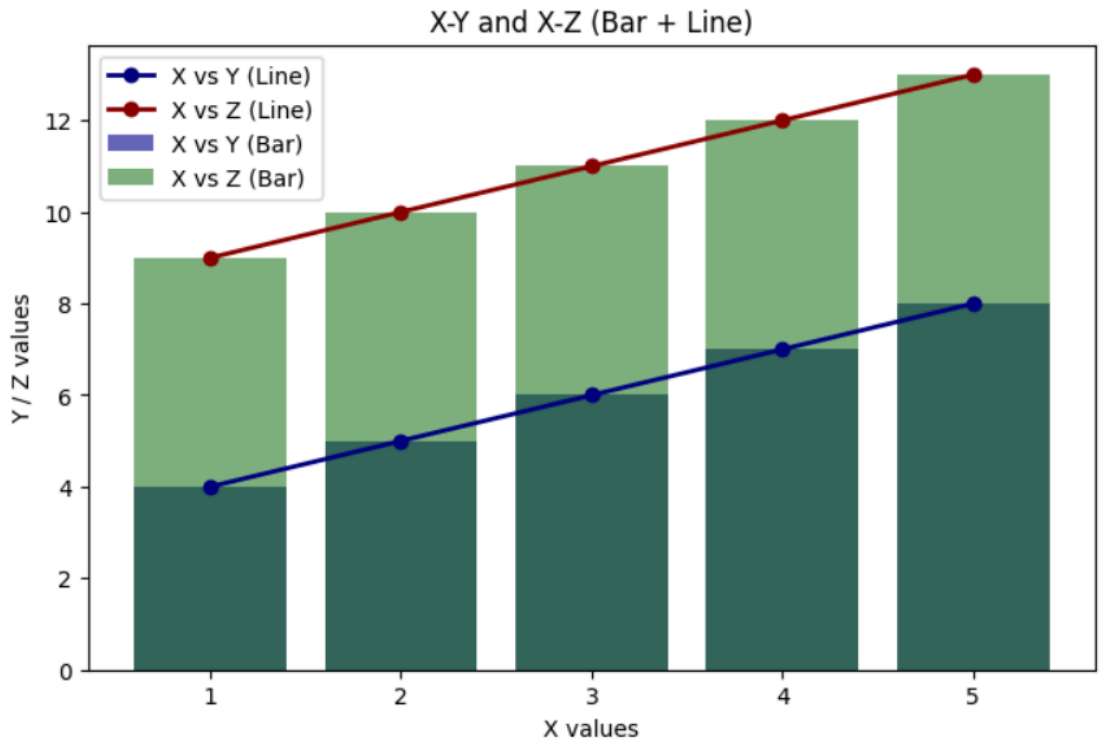
Two datasets (X vs Y and X vs Z) were displayed together using line plots for comparison.



Terminal

8. Combined Bar and Line Graph

Bar charts and line graphs were combined in a single figure to compare multiple datasets visually.



Conclusion

Through this activity, different visualization techniques in Matplotlib were learned and implemented successfully. The task improved understanding of graph plotting, subplot arrangement, and comparison of datasets using various chart types. These visualizations help in better analysis and interpretation of data in Python.